# PIC 16B Final Project Report: Pokemon Type Identification as a Multi-Label Classifcation Task

**Nishanth Tharakan** **Patrick Lerdsuwanrut** **Ajmain Zahin**

12th June 2026

## Contents

# 1. Introduction

Pokémon is a Japanese media franchise, created and owned by the company Game Freak, based on cartoonish creatures by the titular name.

Specifically in the video games and trading card game, different Pokémon have different stats that affect their use in combat, including numerical attribute statistics, abilities, typing (19 categories, Pokémon can have up to 2), and movesets (up to 4 from a pool specific to that Pokémon).

When encountering a Pokémon in the games, knowing the typing of the Pokémon you are facing can be the difference between dealing enough damage to knock a Water-Ground Pokémon out with a single Grass type move, or dealing no damage with an electric type move. However, knowing the typing of all 1025 Pokémon can be a daunting task, especially when all you are provided is an image of the Pokémon (and its name depending on the game).

## 1.1. Problem Statement

Below we define some of the key problems and questions we want to tackle that motivated the creation of this project:

1. Can a model be trained to identify the typing of a Pokémon solely through images of that Pokémon?
2. How does such a model perform on various subgroups of the data, which includes segregation by actual type, generation released, shiny form, etc?
3. What are the best training regimens to get good results across all Pokémon and their various subgroups?
4. If only given training data for some generations of Pokémon (e.g. only providing data on generations 1-7), how does the model perform on the remaining generations (in this case, 8 and 9)?

## 1.2. Project Objectives/Goals

Our main goals for the project are described in some detail below:

1. Determine feasibility for various models to classify Pokémon sprite images.
2. Figure out which models, methodologies, and training data give the best results in terms of the goal stated above, and how the models perform comparative to our own ability to guess Pokémon typing.
3. See what patterns and insights we can gather about Pokémon design choices over generations.
4. What can classifying Pokémon sprites tell us about other multi-label classification tasks?

## 2. Data Collection and Processing

We had to compile two separate datasets to create our training/testing data:

PokeAPI [1] was our source for form and typing information for all Pokémon we are testing. It's a RESTful API containing almost all historical information on every Pokémon, but we will be using only the typing information they provide.

Figure 1: PokeAPI logo

We used the requests Python library to get PokeAPI data, and a shell script to pull the Pokerogue Github Repository, isolate the sprites folder, and delete the rest. (We didn't use Python because we were unsure how to do git pulls and large deletions with Python)


Figure 2: Example of Pokerogue sprite, with a special Pokerogue-exclusive color palette. For our work, we are only using the default color palettes for all Pokémon.

Once the sprite sheets were collected, code was implemented to go through them all and split them into sheets based on individual Pokémon. Then, the code would crop individual sprites from the sheet into separate PNGs for the model to use.

Pokerogue [2] has fan made sprites for all 9 generations of Pokémon (except for Pokémon: Legends Z-A Mega Evolutions, which will not be tested in this dataset) in a consistent pixel art format, which contrasts Game Freak's change to 3d models for their Pokémon sprites since Generation 6 in 2013). While we could have used official Pokémon 3d models from Pokémon Home, using them would present computational difficulties for us as they are very high quality models.

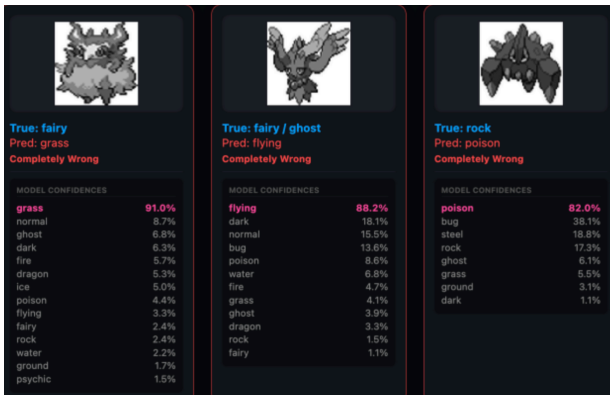


Figure 3: We collect sprite information with a shell script, and type information with the `requests` python library and PokeAPI, and finally combine in when we want to make a dataset for training.

## 2.1. Initial Analysis

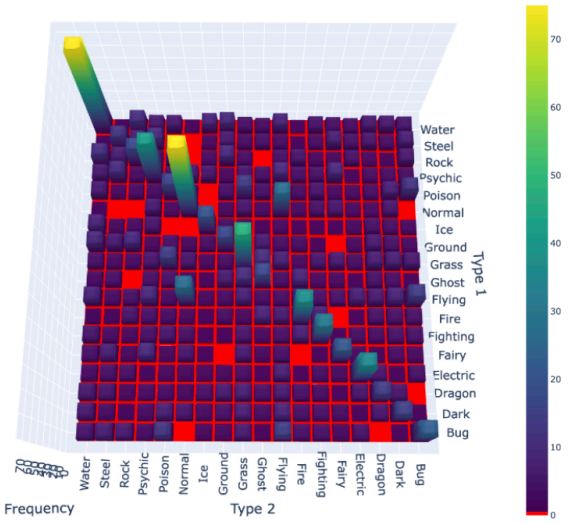


Figure 4: 3D heat map of all 171 type combinations as a symmetric matrix (Water-Flying identical to Flying-Water). Bright colors means more representatives, red means no representatives.

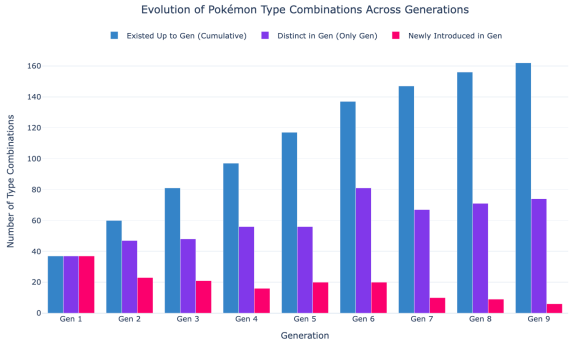


Figure 5: The progression of Pokémon types by generation. Almost every generation introduced 100-200 new Pokémon, but there is a clear trend of later generations making those Pokémon have more distinct typings.

There are $\binom{19}{2} = 171$ distinct Pokémon Types, meaning on average, each typing will have $\frac{1024}{171} \approx 6$ representatives. Even accounting for the 9 unused type combinations we can see that the representation for each type is not particularly good.

At the same time, some type combinations, like Water, Flying, and Normal-Flying, have disproportionally high amounts of representation in the data set, with over 20 representatives each. With this data, we can conclude that it is probably not feasible to have a model identify both types for a given sprite, but we continue the project by also rewarding the model just as much for a partial identification (such as identifying a Water-Flying type as a Water-Grass or mono-Flying type)

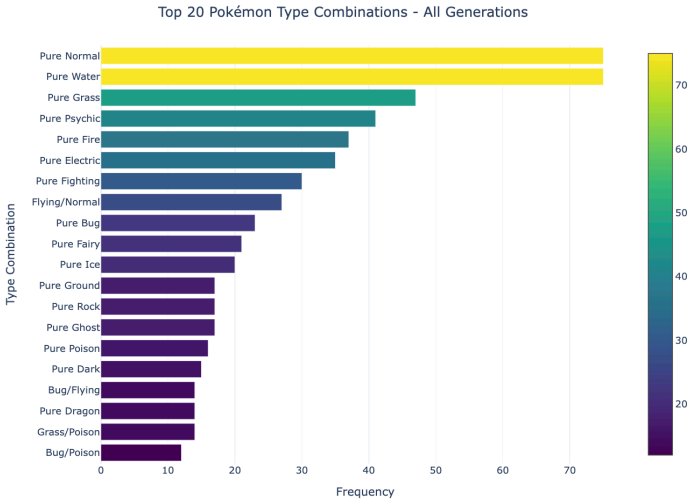


Figure 6: Bar graph of the top 20 type combinations, single types identified as “Pure,” dual types identified with a slash

# 3. Models

## 3.1. Non-Deep Learning Baselines

Three flat-feature classifiers were built to establish lower bounds and show where deep learning is actually needed. All three use the same pipeline:


Figure 7: The pipeline for our non-deep learning classifiers

The **Decision Tree** classifier constructs hierarchical decision boundaries using principal components. While computationally efficient and highly interpretable, it is prone to overfitting and fails to capture complex visual patterns.

The **Random Forest** classifier ensembles 100 decision trees trained on bootstrapped data and feature subsets, utilizing a majority vote. This ensemble approach mitigates variance, yet its capacity remains constrained by the underlying flat pixel representation.

The **Support Vector Machine (SVM)** with a Radial Basis Function (RBF) kernel optimizes a maximum-margin hyperplane in the PCA-reduced feature space, enabling non-linear classification boundaries and establishing the strongest baseline.

Nonetheless, principal components derived from raw pixel inputs serve as inadequate representations for sprite classification. Furthermore, all three baseline architectures are inherently single-label classifiers, rendering them incapable of predicting secondary types and thus constraining their performance scores.

The fundamental limitation of these baselines lies in the 1D flattening of input images. Flattening discards spatial associations between adjacent pixels. Subsequent PCA compression reduces the 12,288-dimensional pixel space to 50 principal components, preserving only global color gradients while completely erasing the high-frequency shapes and textures essential to distinguishing distinct Pokémon typings.

### 3.2. Deep Learning Model Architectures

#### 3.2.1. EfficientNet-B0

We built the initial EfficientNet-B0 pipeline, which became the foundation for the full Classification/ folder. EfficientNet-B0 is a Convolutional Neural Network (CNN) pretrained on ImageNet. The only change from stock EfficientNet was swapping the final layer to output 18 type scores instead of 1000 ImageNet classes:

```
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 18)
```

Everything before that layer keeps its ImageNet weights and trains end-to-end. The model already knows how to detect edges and color patterns before seeing a single Pokémon sprite.


Figure 8: Pipeline of EfficientNet-B0

**Softmax vs. Sigmoid Activation:** We moved from a softmax output layer to a sigmoid activation function to successfully accommodate multi-label (dual-type) predictions. A softmax function constrains the sum of all 18 type probabilities to 1, meaning a strong prediction for a primary type (e.g., Fire) artificially suppresses the probability of a secondary type (e.g., Flying). Conversely, a sigmoid activation evaluates each type independently, allowing multiple classes to simultaneously score high. Accordingly, the loss function is formulated as BCEWithLogitsLoss, treating the task as 18 independent binary classification decisions.

#### 3.2.2. Scratch CNN


Figure 9: Pipeline of Scratch CNN

EfficientNet-B0 was originally trained for identifying real life animals and objects, but we are instead using it to identify cartoon-style characters. To quantify the usefulness of this pretrained model, we also are using an untrained CNN as a competitor, which otherwise works identically to EfficientNet-B0.

### 3.2.3. ViT-B/16


Figure 10: Pipeline of ViT-B/16

Vision Transformers cut images into 16x16 patches and put it through multiple transformer layers using special classification tokens for prediction. These tokens aggregate information from all the image patches and then use it all to make a prediction in the final layer. ViT has the advantage of being able to look at all aspects of an image and make a more calculated guess, and is likely to learn more from the given data and have less inductive bias. Like EfficientNet B0, ViT is also pre-trained on the massive dataset of "ImageNet-21k," also giving it a lot of data to go off of.

Similar to the Vision Transformer, we only altered the final layer, so pre-train weights were applied from the Image-Net weights and trains as well.

For ViT we tried freezing and not freezing the backbone to see how it performs if we attempt to keep the original weights stable vs changing it in our own training loops.

## 4. Training Regimen

For training we followed these steps:

1. Dataset Split & Setup: Generates the split indices, ensuring all sprites for a specific Pokémon ID stay together

```
dataset = PokémonSpriteDataset()
  # Split strategy selection
  if args.split == "stratified":
      train_idx, val_idx, test_idx = gen_stratified_split(
          dataset.index, val_frac=args.val_frac, test_frac=args.test_frac, seed=args.seed
      )
  Model Training Loop
  Initializes the model architecture, weights sampler, and trains using Binary Cross Entropy loss:
  `# Create loader & build model
  sampler = make_weighted_sampler(dataset, train_idx)
  train_loader = DataLoader(Subset(dataset, train_idx), batch_size=args.batch_size, sampler=sampler)
  model = build_model(num_classes=len(TYPES), freeze_backbone=args.freeze_backbone).to(device)

  # Epoch loop with hotswapping transforms
  for epoch in range(1, epochs + 1):
```

```
    dataset.transform = active_train_tf
    train_m = run_epoch2(model, train_loader, criterion, optimizer, device, train=True)

    dataset.transform = active_eval_tf
    val_m = run_epoch2(model, val_loader, criterion, optimizer, device, train=False)
```

2. Checkpoints: If the F1 score for the validation set improved between epochs, we save the new model:

```
if val_m["f1"] > best_f1:
    best_f1 = val_m["f1"]
    torch.save({
        "epoch": epoch,
        "model_state": model.state_dict(),
        "val_f1": best_f1,
        "args": vars(args),
    }, checkpoint_path)
```

3. Inference: Since the model returns a series of confidence levels, we apply some hard coded logic to determine the single type/dual type quality that the model is telling us:

```
for i in range(len(probs)):
    sorted_idx = np.argsort(probs[i])[::-1]
    # Guarantee absolute highest guess
    preds[i, sorted_idx[0]] = 1
    # Predict second type if confidence is within GAP_THRESHOLD
    if (probs[i, sorted_idx[0]] - probs[i, sorted_idx[1]]) < 0.25:
        preds[i, sorted_idx[1]] = 1
```

Metrics Printing & Visualization HTML Gallery Calculates metrics and outputs base64-encoded mistake cards into an interactive HTML report:

### 4.1. Important features:

As discussed in **Section 2.1**, the dataset exhibits significant class imbalance, characterized by a large amount of Water and Normal types, while types such as Ghost, Dragon, and Ice are underrepresented. Without corrective measures, the model would exhibit bias toward the majority classes. To mitigate this, a weighted sampler was implemented, forcing samples from underrepresented classes to appear more frequently in training batches and ensuring consistent exposure to minority class features.

### 4.2. Augmentation Tricks

1. Changed orientations using `transforms.RandomHorizontalFlip()`
2. Shifted images around using `transforms.RandomAffine()`
3. Tried to mitigate overfocusing on color and overfitting by using `transforms.ColorJitter()` and changing things like brightness, contrast, and saturation.

### 4.3. Grayscale Ablation

To evaluate the extent to which the deep learning model relies on color signatures (such as green for Grass, blue for Water, or red for Fire) versus structural shapes and silhouettes, we conducted a grayscale ablation experiment. The input sprites were converted to three-channel grayscale tensors using `GRAYSCALE_TRAIN_TRANSFORM`. This ablation isolates structural and

contour-based visual indicators from color features to evaluate their respective contributions to classification accuracy.

### 4.4. Inference — Gap Threshold

Originally, we preset the amount of labels the model would label an image with; when testing, we would tell the model if it was one or two types, and we would take the top 1 or 2 accordingly. However, this makes the assumption that we would already know how many types a Pokémon would have, which would defeat the purpose of letting the model figure it out.

Instead, we have preset a difference threshold, which utilizes the assumption that if the model thinks a Pokémon has more than one type, those secondary types would have similar confidence values to the primary type it wants to label the image with. As such, if the model's second option's confidence value is within 25% of the first type's, then the image is classified as a dual type:

```
# 1. Always lock in the absolute #1 highest guess
preds[i, sorted_idx[0]] = 1

# 2. Check if the 2nd highest guess is within the gap threshold
if (probs[i, sorted_idx[0]] - probs[i, sorted_idx[1]]) < GAP_THRESHOLD:
    preds[i, sorted_idx[1]] = 1
```

This is how the model actually runs without knowing the answer in advance.

### 4.5. Evaluation Metrics

We reported F1, Precision, and Recall instead of just accuracy. This is because looking at just accuracy does not always give us a proper idea of a model's performance and we can get a better idea of what a model is getting correct by looking at multiple measures such as the ones mentioned before.

Additionally, for the purpose of our project, focusing too heavily on trying to get exact matches is not fruitful. Checking if the model gets at least one correct gives us a better indication of the model's ability to predict types based on typing without punishing it too heavily for dataset and structural limitations.

We define some unique evaluation metrics we judged our model on below:

1. At Least 1 Right: the model predicted at least one correct type
2. 2 Types Right (Dual): among dual-type Pokémon specifically, both types correct
3. 2 Types Right (All): among all Pokémon, both types correct (including single-type counting only as single types)
4. Per-type breakdown: F1/Precision/Recall for each of the 18 types

## 5. Results Analysis

Below we have the results for the Decision Tree, Random Forest, SVM, and Efficient-NetB0. An important thing to consider here is that what we refer to as “Accuracy” is actually partial accuracy where the model gets at least 1 out of 2 types right.

Table 1: Performance Comparison of Model (Naive baseline is picking Mono-Water the whole time as it's the most frequent type)

| Model | Accuracy | F1 Score | Precision | Recall |
|---|---|---|---|---|
| Naive baseline | 14.91% | 1.44% | 0.83% | 5.56% |
| Decision Tree | 18.44% | 12.93% | 14.77% | 12.45% |
| Random Forest | 29.10% | 19.89% | 28.18% | 17.72% |
| SVM | 31.15% | 19.63% | 24.43% | 18.56% |
| Grayscale CNN | 20.23% | 9.99% | 9.68% | 13.55% |
| Scratch CNN | 38.15% | 31.61% | 31.92% | **35.85%** |
| **EfficientNet-B0** | **53%** | **33.09%** | **33.59%** | 35.09% |

While we did also test EfficientNet-V2 and ViT-16, we did not create or look at detailed metrics for them as the base F1 score during training and validation was always significantly lower across multiple runs, so we determined it not to be worthwhile in analyzing further. For reference, Efficient-NetV2 and ViT-16 never managed to get to an F1 score of above 30% no matter what we tried, while Efficient-NetB0 was normally above that benchmark. One interesting thing about ViT however, was that it had a much slower rate of improvement compared to EfficientNet and took around 150 epochs to reach its max F1, compared to 30-40 for the other models.

As can be seen, the EffientNet-B0 performs the best, while the Scratch CNN isn’t far behind. Both have the advantage of being models that preserve shape when running, unlike the other models, who all do some level of compression that doesn’t take into account that their data is a 2D matrix instead of a 1D column vector.

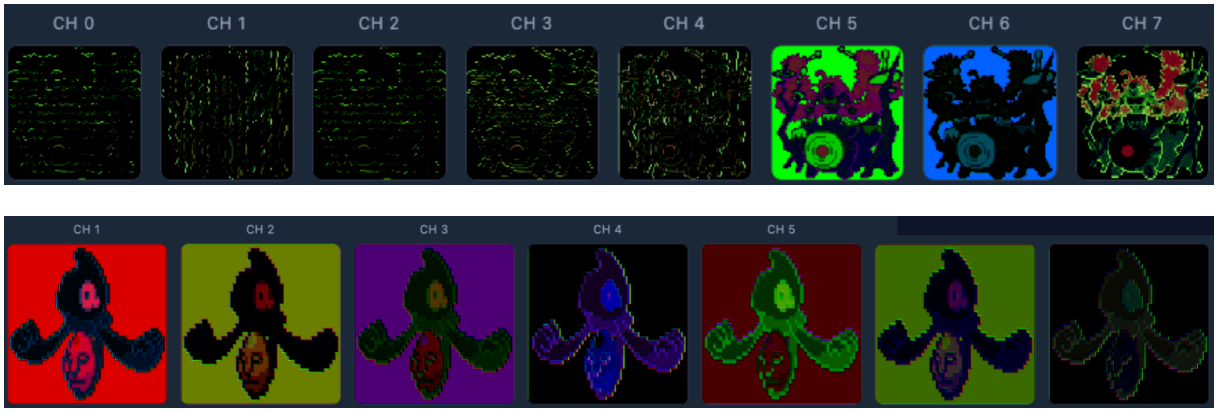


Figure 11: Convolution comparision with Efficient-NetB0 vs Scratch CNN

The visulization of Efficient-NetB0′s convolutional layers compared to the scratch CNN’s also provided some insight on the performance difference. Scratch CNN’s layers really just look like random patterns, except for the last one that looks like a border detector. In contrast, Efficient-NetB0′s layers look purposely build to scan various parts of the image, which each

layer picking up something new, which could explain its better performance (through having more information to use.

Another intersting note about the CNNs is how Efficient-NetB0′s signal charts have very little red on them, while Scratch CNN's signal oput is all over the color spectrum. We believe this is a result of the camera-based training data Efficient-NetB0 was originally trained on, giving it a bias for looking for green and blue hues rather than the whole spectrum, and this could be an important factor in why certain types are detected better than others.

### 5.1. Best Ever VS Average Metrics

A very important highlight of our analysis was how varied our results were, even when we did not change anything. For example, when running our EfficientNet-B0 model multiple times, on the exact same parameters and conditions, we got vastly different results ranging from a partial accuracy score of 47% to a best-ever score of 66%. A more detailed overview of our metrics is seen below.

For our average results we observed:

- At Least 1 Right: 53%
- 2 Types Right (All): 2%
- 2 Types Right (Dual): 3.73%

Table 2: Typical results from the models we generate

| Type | Grass | Water | Fire | Normal | Rock | Bug | Flying | Ghost | Poison |
|---|---|---|---|---|---|---|---|---|---|
| **F1** | 0.6824 | 0.496 | 0.4412 | 0.4211 | 0.4 | 0.3243 | 0.3077 | 0.2791 | 0.2667 |
| **Precision** | 0.5686 | 0.3735 | 0.3409 | 0.3478 | 0.4444 | 0.2927 | 0.381 | 0.2857 | 0.25 |
| **Recall** | 0.8529 | 0.7381 | 0.625 | 0.5333 | 0.3636 | 0.3636 | 0.2581 | 0.2727 | 0.2857 |

,

| Type | Ground | Psychic | Dragon | Ice | Electric | Dark | Fighting | Fairy | Steel |
|---|---|---|---|---|---|---|---|---|---|
| **F1** | 0.2553 | 0.2319 | 0.2222 | 0.1935 | 0.1765 | 0.15 | 0.1395 | 0.0952 | 0.093 |
| **Precision** | 0.3 | 0.25 | 0.2857 | 0.1875 | 0.1875 | 0.12 | 0.1667 | 0.0714 | 0.1429 |
| **Recall** | 0.2222 | 0.2162 | 0.1818 | 0.2 | 0.1667 | 0.2 | 0.12 | 0.1429 | 0.069 |

For our best results we observed:

- At Least 1 Right: 66.47% (13% naive baseline)
- 2 Types Right (All): 21.97% (Single as Single, Dual as Dual. The Exact Accuracy)
- 2 Types Right (Dual): 40.86% (Out of only dual-type Pokémon)

Table 3: Results from our best model ever (anomaly)

| Metric | Psych. | Grass | Ice | Dark | Rock | Dragon | Water | Ghost | Norm. |
|---|---|---|---|---|---|---|---|---|---|
| F1 | 0.8000 | 0.7826 | 0.7778 | 0.7500 | 0.6667 | 0.6471 | 0.6071 | 0.5926 | 0.5714 |
| Precision | 0.8235 | 0.7200 | 0.7000 | 0.8571 | 0.9000 | 0.5789 | 0.5152 | 0.6154 | 0.5000 |
| Accuracy | 0.7778 | 0.8571 | 0.8750 | 0.6667 | 0.5294 | 0.7333 | 0.7391 | 0.5714 | 0.6667 |

,

| Metric | Fight. | Steel | Fire | Ground | Electr. | Bug | Flying | Fairy | Pois. |
|---|---|---|---|---|---|---|---|---|---|
| F1 | 0.5455 | 0.5405 | 0.5333 | 0.5161 | 0.5000 | 0.4571 | 0.4138 | 0.3333 | 0.1000 |
| Precision | 0.7500 | 0.6667 | 0.4000 | 0.7273 | 0.5000 | 0.4444 | 0.6000 | 0.4000 | 0.0909 |
| Accuracy | 0.4286 | 0.4545 | 0.8000 | 0.4000 | 0.5000 | 0.4706 | 0.3158 | 0.2857 | 0.1111 |

Such variance seems really surprising considering nothing really changed except how the dataset was split during the train-test-split. This highlights the fact that without a large and diverse dataset, there is a big risk of result variance just from what data is used to train and what is used to test.

It is also interesting to note that even accounting for the variance, some types just seem to be easier for the model to predict such as grass, water, and rock, whereas other types like poison, fairy, and ghost seem to have much lower accuracies and precisions. This can probably be attributed to certain types (like typical elemental types) having more telling color-based designs, whereas the more exotic types, like Dragons, Ghosts, and Fairies, have more abstract designs based on famous monsters or folktales.

### 5.2. Results using different augmentations and sample sizes

Most augmentations did not actually lead to any meaningful changes in our average metrics. The most surprising one was perhaps ColorJitter where changing brightness, contrast, and even saturation still yielded nothing significantly different. We hypothesize that due to the existing weights in the pre-trained model, perhaps such changes on a relatively much smaller dataset do not have too much of an effect. However, we see later that completely removing color does have an effect, so perhaps even with moderate levels of saturation, the model can make out the broader colors in the image.

The relevant line of code is given below:

```
transforms.ColorJitter(brightness=0.15, contrast=0.15)
```

Moving on to sample sizes, we observed that having a bigger variety of data was usually helpful. Using the full sample size of unique Pokémon consistently gave us better results. After using the original versions of each Pokémon, we also added variants (specifically those without new typings, the ones with new typings are always in the dataset) of the different

Pokémon to increase our dataset variety further in a meaningful way. However, adding too many variants would sometimes lead to a cluster of the same Pokémon being tested which skewed results, so we limited variants to 3 per Pokémon.

While having variety like this helped, trying to artificially increase sample size by just reusing almost identical sprites did not change our results. Thus, we can conclude that in terms of sample size, the uniqueness of the samples seems to matter more than just the actual quantity.

### 5.3. Greyscale analysis

When we applied greyscaling, we saw a massive drop in accuracy to only 20% partially correct, which was expected. However, looking closely at the results below, we can see that the model actually does seem to be able to read features well.

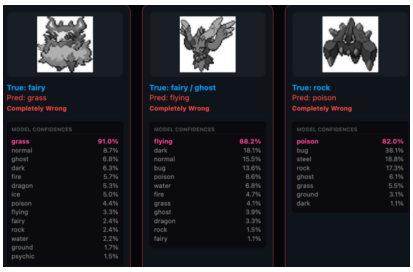


Figure 12: Greyscale analysis, as seen in our in-class presentation

The model most likely predicts the first one as Grass because you can actually see the Pokémon within a patch of grass. Similarly, the second Pokémon is assigned Flying because it appears to have wings. While the third seems to be off, it does kind of look like a small bug, especially one with needles/ spiky features, which are common for Poison types.

### 5.4. Generation-Wise Split Analysis (Incomplete due to lack of time for adequate training)

For the final part of our analysis we tested how the results changed if we trained and tested or model on different generations of Pokémon. For this purpose, 4 segmentations were tried:

- Segment 1: Training done on gen 1-3, tested on 4-6.
- Segment 2: Training done on gen 1-8, tested on 9.
- Segment 3: Training done on gen 3-9, tested on 1-2.
- Segment 4: Training done on gen 2-6, tested on 1.

The purpose of Segment 1 was to analyze how the model performs given the same number of generations for training and testing. This yielded really poor results, with F1 falling by 10% and accuracy also falling by a few percentage points.

Segment 2 is really similar to just training on the entire dataset, but the model is not given the last generation. This made no difference as the model seems to have gotten enough training to predict generation 9 and the small number of Pokémon eliminated makes little difference.

Segment 3 is designed to check how the model predicts the simpler designs of early generations after being trained on more complex ones. It seemed to be performing very slightly better by a few percentage points, although given the possibility of the variance we cannot say for certain that performance was surely better.

Segment 4 went back to being the same as the added benefit from being tested on simpler types, and being trained on closer generations, was cancelled out by having a lower dataset.

For this section, we did not have ample time to conduct a detailed analysis given the data limitations and lack of means to determine causal inference, and thus no conclusive statements can be made without further research. However, our analysis so far suggests that there may actually be some difference in design complexity, and thus model performance, across generations. Given more time, like presenting this at a PIC talk, we would have discussed these ideas at length.

## 6. Conclusion

Overall, our models seemed to perform relatively well given how hard the task of predicting types of just design actually is. When we started this project, we took inspiration from 2 projects done before. First was Tariq, a then student at Stanford who made this project for his Deep Learning Class [3]. Building on top of his idea of using a CNN, our unique contributions in the dataset we used, the way we turned model confidences to predictions, and other augmentations, yielded a much better F1 score of around 33% compared to his 24%. Our second reference point was a project on this topic by Garrett Hardin, which we came across on a post on Medium [4]. Unfortunately, he mainly tested and trained his model on just gen 1 which means comparing our results to his is not meaningful. However, we have started to and want to build upon his generation wise splitting further to see what more we can learn.

This project displayed the importance of always being statistically rigorous when conducting reserach, specifically showcasing what can happen with a flawed dataset or a flawed pipeline. If we didn't repeat trials we may new have noticed the high variance between different splits, or the anomalously successful trial where the stars aligned for a very accurate model. Visualization also proved to be an important part of analysing results as without it we would have not realized Efficient-NetB0′s biases towards greens and blues.

While Pokémon identification may not be a very useful task in the real world, other multi-label classifcation tasks, like injury detection from X-rays or content moderation online, are important for society to help the sick or maintain order in an online space. These practices, while techinically different tasks, all are united under the common feature of assigning multiple labels to one thing, which means these tasks can also face the same issues we ran into, like lack of data variety, or using a pretrained guide that comes with its own biases.

## 7. Member Contributions

- Nishanth:
  - Data acquisition: wrote all scripts related to getting Pokerogue and PokeAPI data (Patrick modified them to allow multithreading)
  - Suggested the EfficientNet-B0 model; implemented the Scratch CNN.
  - Cowrote analysis with Ajmain, wrote about the results of our project that note the importance of certain scientific practices.
- Ajmain:
  - Implemented the ViT-B/16 architecture;
  - Implemented the inference gap threshold,

- Selected and reported evaluation metrics.
- Cowrote analysis with Nishanth

- Patrick:
  - Developed the flat-feature classifiers (Decision Tree, Random Forest, SVM) and the initial EfficientNet-B0 pipeline;
  - Designed the weighted sampler and co-designed the inference gap threshold;

## 8. References:

PokéRogue PokeAPI

## Bibliography


[1] “PokeAPI.” [Online]. Available: https://pokeapi.co/

[2] “PokeRogue Sprites Credits.” [Online]. Available: https://wiki.pokerogue.net/credits:sprites

[3] T. Zahroof, “ "What's That Pokemon?" .” [Online]. Available: https://cs230.stanford.edu/projects_spring_2019/reports/18664574.pdf

[4] G. Hardin, “ Pokemon Identifier and Classifier Using Convolutional Neural Networks .” [Online]. Available: https://medium.com/@hajin.kim/pokemon-identifier-and-classifier-using-convolutional-neural-networks-a8ec9d646c4